



Special Course in Software Engineering '26

Project – Plan and Develop

Important

- Work in groups
 - Therefore, you must sit with your group members
 - Communicating over WeChat or similar application is not acceptable
 - If you don't like this, then you are free to leave, and you'll be marked absent

Check your current requirements

Make sure the requirements.json artifact has the following structure and content:

```
{  
  "goal": "Your own goal that the Qwen model produced",  
  "allowed_actions": ["FORWARD", "LEFT", "RIGHT", "STOP"],  
  "safe_stop": true,  
  "avoid_obstacles": true  
}
```

The important part is that the structure is as shown above, and of course the **allowed_actions** have only those values and nothing extra

Planner Agent

- **Analyst Goal:** Decide what the software must achieve
- **Planner Goal:** Decide how the software should behave
- As you've learned in the last lecture, in a multi-agent system, the output from one agent becomes the context for the next agent.
- Therefore, the **Planner Agent** must receive the validated requirements that you got at the end of the last project ask
- It should not receive the the entire Analyst conversation: **CONTEXT ISOLATION**

Planner Agent

Follow the instructions in **planner_agent.py**

- Create a suitable system prompt
- A function **run_planner()** that takes the requirement artifact, runs it via Qwen, gets the output and validate it using **validate_plan()**

Run Planner

Follow the instructions in **run_planner.py**

- This is similar to the **run_analyst()** you created for the Analyst agent earlier.
- Add the logic to parse the requirements from the requirements.json artifact, pass it to **planner_agent.py**, get the validated result from that agent, and save it into **plan.json**
 - **plan.json** will be the artifact that the **Developer Agent** will use
- The **plan.json** should contain
 - A strategy for robot navigation
 - A list of decisions the robot can take during navigation
 - The point at which the robot must stop

Developer Agent and Run Developer

The instructions for creating a Developer Agent is similar to the Planner Agent

- Create **developer_agent.py**, which includes the System Prompt, a function to validate the output from the agent, which must be a proper **Python code**.
 - The code must contain some function that makes sense for your robot's navigation
- Create **run_developer.py**, which has similar logic as **run_planner.py** and **run_analyst.py**
 - This file stores the generated code in a file called **navigation_logic.py** file

Submission

- Update your earlier GitHub repository, which must have the name **scse_agentic_se_groupname**
 - Replace **groupname** with your actual group name.
- All the files mentioned in this instruction must be available in the repository
- Submit only the GitHub repository link to Moodle by **September 25, Midnight**

That's all for this practice